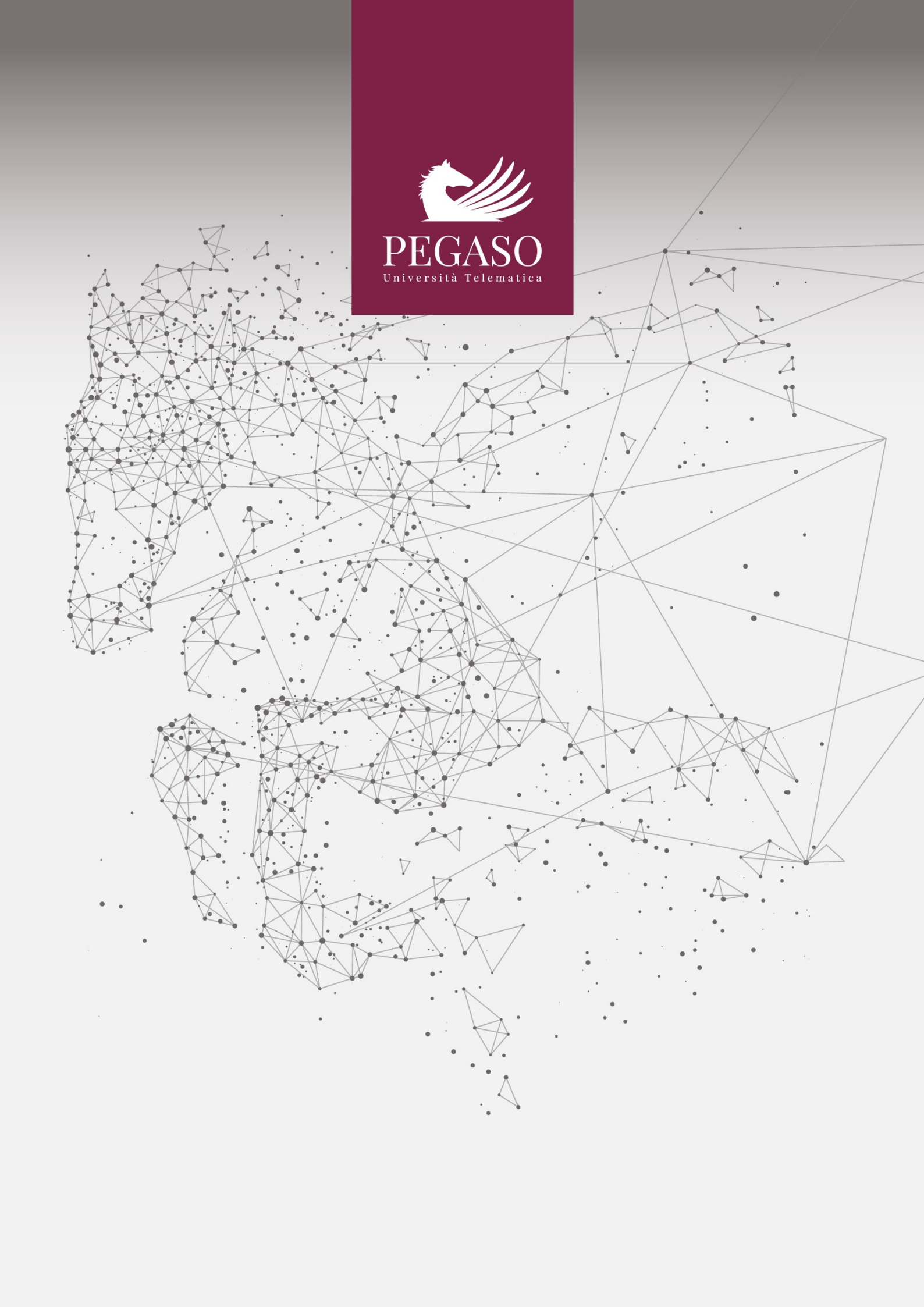

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. INTRODUZIONE ALLE LISTE 4
3. METODI DELLE LISTE 12
4. ELABORAZIONE DELLE LISTE 16
5. LISTE BIDIMENSIONALI 18
6. TUPLE 21
BIBLIOGRAFIA 22

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si introducono due strutture dati molto utilizzate nei programmi scritti in Python: le liste e le tuple. In particolare, dopo un'introduzione alle liste, si presenteranno le operazioni che Python offre sulle liste per la risoluzione di problemi che necessitano di sequenze di dati. Verranno forniti esempi di elaborazione delle liste e introdotte anche le liste bidimensionali. Al termine, si introdurranno brevemente le tuple e si illustreranno le differenze principali tra liste e tuple.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Introduzione alle liste

Una **sequenza** è un oggetto che contiene più elementi. In una sequenza, infatti, gli elementi sono memorizzati in sequenza, uno dopo l'altro. Python fornisce diversi tipi di sequenze: in questa lezione introdurremo due strutture dati, le **liste** e le **tuple**. La differenza principale tra liste e tuple è che le liste sono modificabili, mentre le tuple non sono modificabili.

Una **lista** è un oggetto che contiene diversi elementi il cui formato è il seguente:

```
lista = [elemento1, elemento2, ...]
```

Gli elementi di una lista sono ricompresi tra parentesi quadre e sono separati da virgole. È importante sottolineare che la lista può contenere elementi di tipi diversi. Ne segue, quindi, che la lista non è necessariamente omogenea. La funzione **print**() può essere utilizzata per mostrare l'intera lista attraverso la visualizzazione di default. Si riporta di seguito il codice sorgente che inizializza tre liste (una omogenea di numeri interi, una omogenea di stringhe, una non omogenea di numeri e stringhe) e le stampa a schermo con la visualizzazione di default:

```
lista_num = [1, 2, 3, 4, 5]
lista_str = ["verde","giallo","blu","rosso"]
lista_mista = [1, "giallo", "blu", 4, 5]

print(lista_num)
print(lista_str)
print(lista_mista)
```

Il risultato dell'elaborazione è il seguente:

```
[1, 2, 3, 4, 5]
['verde', 'giallo', 'blu', 'rosso']
[1, 'giallo', 'blu', 4, 5]
```

La funzione **list**(), oltre a restituire una lista vuota se non vengono passati dei parametri, può convertire alcuni tipi di oggetto in liste.

Al fine di semplificare l'inizializzazione delle liste, Python mette a disposizione anche l'**operatore di ripetizione ***

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'operatore di ripetizione crea copie multiple di una lista e le unisce insieme come nell'esempio successivo in cui si inizializzano due liste: la prima ripetendo e accodando la lista composta da un solo elemento 0 per 5 volte (totale elementi lista = 5) e la seconda ripetendo e accodando la lista composta da due elementi 0 e 1 per 5 volte (totale elementi lista = 10):

```
lista_zero = [0] * 5
lista_bin = [0,1] * 5

print("lista zero =",lista_zero)
print("lista bin =",lista_bin)
```

Il risultato dell'elaborazione è il seguente:

```
lista zero = [0, 0, 0, 0, 0]
lista bin = [0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
```

La lista in Python è un **oggetto iterabile** e si può usare, dunque, un ciclo for per iterare sulla lista. Di seguito si riporta un esempio di utilizzo del ciclo for per ottenere una visualizzazione della lista diversa da quella di default:

```
lista = [1,2,3,4,5,6,7]

for x in lista:
    print("elemento", x)
```

Il risultato dell'elaborazione è il seguente:

```
elemento 1
elemento 2
elemento 3
elemento 4
elemento 5
elemento 6
elemento 7
```

Per l'accesso alla lista è possibile utilizzare la tecnica dell'**indicizzazione (indexing)**. L'**indice (index)** è un numero che specifica la posizione di un elemento in una lista. Nell'esempio successivo viene inizializzata una lista con 5 lettere. Considerando che le posizioni delle lettere nella lista cominciano da zero, avremo un intervallo di validità dell'indice tra 0 e 4 compresi. Viene richiesto di visualizzare a schermo l'elemento con indice 4 corrispondente, quindi, al 5° elemento:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
lista = ['a','b','c','d','e']
print(lista[4])
```

A schermo, dunque, verrà visualizzata la lettera e:

```
e
```

Volendo analizzare le caratteristiche dell'indice è possibile affermare che:

- l'indice permette l'accesso ad uno specifico elemento della lista;
- come anticipato, l'indice del primo elemento della lista è 0;
- l'indice dell'n-esimo elemento è n-1;
- indici negativi identificano posizioni relative alla fine della lista;
- l'indice -1 identifica l'ultimo elemento, l'indice -2 identifica il penultimo, etc;
- se l'indice non è valido (al di fuori della dimensione della lista) viene sollevata **eccezione IndexError**.

Per conoscere la dimensione di una lista Python mette a disposizione la **funzione len**() che restituisce la lunghezza, espressa come numero intero, della sequenza della lista. Nel codice seguente, si riporta un esempio dell'uso della funzione len():

```
lista = ['a','b','c','d','e']
l = len(lista)
print("La lunghezza della lista di caratteri è",l)
```

Il risultato dell'elaborazione è:

```
La lunghezza della lista di caratteri è 5
```

Poiché la funzione len() restituisce il numero degli elementi, l'indice dell'ultimo elemento della lista sarà len(list)-1. La funzione len() può, inoltre, essere usata per evitare l'eccezione di tipo **IndexError** quando si accede ad una lista come mostrato nel codice seguente:

```
i = 10
if (i<len(lista)):
    print(lista[i])
else:
    print("Indice non corretto. Dimensione lista =", len(lista))
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato dell'elaborazione è il seguente:

```
Indice non corretto. Dimensione lista = 5
```

Alternativamente bisogna gestire l'eccezione IndexError come riportato nel codice seguente:

```
i = 10
try:
    print(lista[i])
except IndexError:
    print("Indice non corretto. Dimensione lista =", len(lista))
```

Il risultato dell'elaborazione sarà lo stesso del caso precedente:

```
Indice non corretto. Dimensione lista = 5
```

La lista è una **sequenza mutabile** e, quindi, i valori nella sequenza possono essere cambiati.

È possibile assegnare ad un elemento della sequenza un nuovo valore. Si ricorda, però, che l'indice deve essere valido per evitare l'eccezione IndexError. Un esempio di modifica di un elemento della sequenza potrebbe essere il seguente, in cui si sostituisce il primo elemento della lista con la lettera z:

```
lista[0] = 'z'
```

Si forniscono ora tre esempi di esercizi con le liste. Il primo esempio è quello di creare una lista di 10 elementi da 1 a 10. Nella funzione main(), si inizializza una lista di 10 elementi a zero. Successivamente, per ciascun valore dell'indice i nel ciclo for (da 0 a 9 inclusi), si inserisce nella lista il valore di i+1 (da 1 a 10 inclusi).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Viene, infine, stampata la lista:

```
def main():
    conta = [0] * 10

    for i in range(10):
        conta[i] = i+1

    print(conta)


main()
```

Il risultato dell'elaborazione del codice precedente sarà:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Il secondo esempio di programma con le liste è quello di creare una lista di 10 elementi con numeri inseriti dall'utente. Come nell'esempio precedente, nella funzione main() viene inizializzata la lista conta a 10 elementi tutti a zero. Con un ciclo for, quindi, si chiedono in input, per ciascun valore dell'indice i da 0 a 9, dei valori interi che l'utente inserirà. Si procede, poi, a stampare la lista conta. Si noti il prompt della funzione input: si tratta di concatenazione di stringhe in cui viene fornita anche l'indicazione di quale elemento si deve inserire 1°, 2°,…, 10°:

```
def main():
    conta = [0] * 10

    for i in range(10):
        conta[i] = int(input("Inserire "+str(i+1)+"° elemento: "))

    print(conta)


main()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato dell'elaborazione sarà:

```
Inserire 1° elemento: 1
Inserire 2° elemento: 2
Inserire 3° elemento: 3
Inserire 4° elemento: 4
Inserire 5° elemento: 5
Inserire 6° elemento: 6
Inserire 7° elemento: 7
Inserire 8° elemento: 8
Inserire 9° elemento: 9
Inserire 10° elemento: 10
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Il terzo esempio, infine, è quello di creare una lista di 10 elementi con numeri pari consecutivi. L'inizializzazione della lista nella funzione main() è la stessa del caso precedente. Con il ciclo for, invece, si itera sulla lista al fine di inserire come elementi il valore di (i+1)*2. I valori di i varieranno nell'intervallo [0-9], quindi, i valori di i+1 varieranno nell'intervallo [1-10] e i valori di (i+1)*2 saranno i numeri pari da inserire nella lista. Infine, la lista viene visualizzata a schermo tramite la funzione print():

```
def main():
    conta = [0] * 10

    for i in range(10):
        conta[i] = (i+1)*2

    print(conta)


main()
```

La lista conterrà i seguenti numeri pari:

```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

L'**operatore +** applicato alle liste permette di concatenarle. Non è possibile, però, concatenare una lista con altri tipi di dato (es. numero). Nell'esempio seguente, si inizializzano due liste (una di lettere e una di numeri) e, successivamente, si concatenano nella lista lista_tot.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Infine, viene visualizzato a schermo il contenuto della lista lista_tot:

```
lista1 = ['a','b','c','d','e']
lista2 = [1,2,3,4,5,6,7,8]

lista_tot = lista1+lista2
print(lista_tot)
```

La lista lista_tot a seguito dell'elaborazione sarà la seguente:

```
['a', 'b', 'c', 'd', 'e', 1, 2, 3, 4, 5, 6, 7, 8]
```

Python offre, inoltre, la possibilità di recuperare **parte di una lista (sottolista o slice)**, cioè un intervallo di elementi presi da una sequenza. Nello **slicing della lista**, l'intervallo viene definito da due indici: *inizio* (incluso) e *fine* (escluso). Se *inizio* non è specificato, l'indice di partenza è zero mentre se *fine* non è specificato, indice di fine è len(lista). Il formato per lo slicing in Python è il seguente:

```
sublista = lista[inizio:fine]
```

Volendo fornire un esempio, il codice sorgente che segue estrae una slice della lista che parte dall'indice 2 incluso (terzo elemento) e indice 4 escluso:

```
lista = ['a','b','c','d','e']
inizio = 2
fine = 4
sublista = lista[inizio:fine]
print("Slice =",sublista)
```

La visualizzazione a schermo della funzione print() sarà la seguente:

```
Slice = ['c', 'd']
```

Per sapere se un elemento in particolare è presente in una lista, è possibile utilizzare l'**operatore in**. Con l'operatore in è possibile creare delle condizioni che hanno il seguente formato:

```
elemento in lista
```

Tale condizione, come tutte le condizioni, restituisce un valore logico True se l'elemento è nella lista o False se non lo è. Oltre all'operatore in, esiste anche l'**operatore not in**.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'obiettivo dell'esempio seguente è quello di controllare una lista di invitati. In particolare, nella funzione main() viene inizializzata la lista_invitati, viene richiesto il nome da ricercare e, in base alla presenza o meno del nome tra gli invitati, viene visualizzato a schermo un messaggio diverso:

```
def main():
    lista_invitati = ["Antonio", "Francesca", "Rosa", "Paolo"]

    nome = input("Inserire nome invitato/a: ")
    if nome in lista_invitati:
        print(nome," è tra gli invitati")
    else:
        print(nome," non è tra gli invitati")


main()
```

Ipotizzando di inserire in input il nome Antonio (presente nella lista), il risultato dell'elaborazione è:

```
Inserire nome invitato/a: Antonio
Antonio  è tra gli invitati
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Metodi delle liste

La lista è un oggetto che espone diversi **metodi**, che ricordiamo essere delle funzioni che appartengono alla lista ed agiscono su di essa. I metodi principali di una lista sono:

- **append(elemento)** permette di aggiungere un elemento in coda alla lista;
- **index(elemento)** permette di determinare l'indice corrispondente dell'*elemento* nella lista. Tale metodo restituisce un numero intero della prima occorrenza dell'elemento nella lista. Se l'elemento non è nella lista solleva **eccezione ValueError**. È possibile evitare l'eccezione con l'uso dell'**operatore in**;
- **insert(indice, elemento)** usato per inserire l'elemento nella posizione *indice* della lista;
- **sort()** usato per ordinare gli elementi in ordine crescente;
- **remove(elemento)** rimuove la prima occorrenza dell'*elemento* nella lista;
- **reverse()** inverte l'ordine degli elementi nella lista.

Per illustrare i metodi appena descritti, si fornisce un programma di esempio. In particolare, nella funzione main() viene inizializzata una lista vuota. Con un ciclo for si procede ad inserire in coda un numero intero nella lista. Attraverso un ciclo while nidificato al ciclo for, si evita la ripetizione di numeri già presenti nella lista. Volutamente, per scopi didattici, si chiede di stampare la lista dopo ogni inserimento di numeri nella lista stessa.

Si procede, poi, con:

- un ordinamento e successiva stampa della lista;
- un'inversione dell'ordine degli elementi della lista con successiva stampa;
- la rimozione del numero 1 dalla lista dei numeri;
- l'inserimento del numero 1 nella posizione con indice 4 (quinta posizione).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def main():
    lista = []

    for i in range(5):

        num = int(input("Inserire "+str(i+1)+"° valore "))
        while num in lista:
            num = int(input("Inserire "+str(i+1)+"° valore "))
        lista.append(num)

        print(lista)

    lista.sort()
    print("Lista ordinata =",lista)

    lista.reverse()
    print("Lista reverse =",lista)

    lista.remove(1)
    print("Lista dopo rimozione elemento 1 =",lista)

    indice = 4
    lista.insert(indice,1)
    print("Lista dopo inserimento elemento 1 in posizione 4 = ",lista)

main()
```

L'elaborazione del programma produrrà il seguente output (si noti che, volutamente, a volte viene inserito in input un numero già presente nella lista):

```
Inserire 1° valore 4
[4]
Inserire 2° valore 4
Inserire 2° valore 4
Inserire 2° valore 5
[4, 5]
Inserire 3° valore 1
[4, 5, 1]
Inserire 4° valore 1
Inserire 4° valore 1
Inserire 4° valore 3
[4, 5, 1, 3]
Inserire 5° valore 2
[4, 5, 1, 3, 2]
Lista ordinata = [1, 2, 3, 4, 5]
Lista reverse = [5, 4, 3, 2, 1]
Lista dopo rimozione elemento 1 = [5, 4, 3, 2]
Lista dopo inserimento elemento 1 in posizione 4 =  [5, 4, 3, 2, 1]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Oltre ai metodi appena illustrati, molto utili sono anche l'**istruzione del** e le **funzioni min()** e **max()**.

L'**istruzione del** rimuove un elemento da una specifica posizione come nell'esempio di codice seguente (viene eliminato l'elemento di indice 2, cioè il terzo elemento 'c'):

```
lista = ['a','b','c','d','e']
print(lista)
del lista[2]
print(lista)
```

Segue la visualizzazione a schermo della lista prima e dopo la rimozione dell'elemento di indice 2:

```
['a', 'b', 'c', 'd', 'e']
['a', 'b', 'd', 'e']
```

Le **funzioni min()** e **max()**, invece, restituiscono, rispettivamente il minimo e massimo della lista. Nel codice che segue, si inizializza una lista e si chiede di stampare a schermo il valore minimo e il valore massimo della lista:

```
lista = [1,2,3,4,5]
print("Valore minimo della lista:",min(lista))
print("Valore massimo della lista:",max(lista))
```

L'output dell'elaborazione del codice precedente è:

```
Valore minimo della lista: 1
Valore massimo della lista: 5
```

Infine, per la copia di una lista in un altro oggetto è possibile procedere con due soluzioni.

La prima è quella di creare una lista vuota e usare un ciclo for per aggiungere un elemento alla volta alla nuova lista, come mostrato nel codice seguente:

```
lista = [1,2,3,4,5]

lista_new = []
for elem in lista:
    lista_new.append(elem)
print(lista_new)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La seconda soluzione è quella di creare una lista vuota e concatenare la vecchia lista con la nuova, come mostrato nel codice seguente:

```
lista = [1,2,3,4,5]

lista_new = list()
lista_new = lista_new+lista
print(lista_new)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Elaborazione delle liste

Al fine di illustrare un esempio di elaborazione con le liste leggermente più complesso di quello illustrato nel paragrafo precedente, si chiede di sviluppare un programma organizzato in funzioni che permetta di offrire le seguenti opzioni su una lista di votazioni:

- lettura di valori di votazioni da file di testo;
- inserimento di tali valori in una lista;
- calcolo del totale dei voti;
- calcolo della media dei voti.

Il formato del file di testo che comprende tutte le votazioni è il seguente:

votazioni.txt

```
7
10
8
9
7
6
5
8
9
10
```

Nel codice sorgente che segue si dichiarano tre funzioni main(), lettura_voti() e calcolo_totale(). Nella funzione main() si procede all'apertura in lettura del file "*votazioni.txt*".

Successivamente, il main passa l'oggetto file (gestore) *infile* alla funzione lettura_voti() che restituirà la lista delle votazioni lette dal file stesso. Verrà, dunque, chiamata la funzione calcolo_totale(), a cui verrà passata la lista *voti*, che restituirà la somma dei voti contenuti nella lista. Infine, il main calcolerà la media (si noti il casting temporaneo a float del totale dei voti per ottenere un risultato in virgola mobile) dato dalla divisione tra il totale dei voti e il numero dei voti nella lista, Infine, la media verrà visualizzata a schermo, avendo cura di formattare la media con due cifre decimali.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
#funzione di lettura dei voti da file
#ed inserimento nella lista
def lettura_voti(infile):
    voti = list()

    for riga in infile:
        voti.append(int(riga.rstrip('\n')))
    return voti

#funzione di calcolo del totale dei voti
#nella lista
def calcolo_totale(voti):
    acc = 0
    for elem in voti:
        acc+=elem
    return acc


def main():
    infile = open('votazioni.txt','r')
    voti = lettura_voti(infile)

    tot = calcolo_totale(voti)
    media = float(tot)/len(voti)
    print("La media dei voti è",format(media,'.2f'))


main()
```

Il risultato dell'elaborazione sarà:

```
La media dei voti è 7.90
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Liste bidimensionali

Le **liste bidimensionali** sono liste annidate cioè una lista contenente delle liste come elementi. Le liste bidimensionali possono essere pensate come delle matrici (con righe e colonne).

Ovviamente, per poter lavorare su liste bidimensionali è necessario utilizzare due indici e, solitamente, cicli annidati per l'elaborazione.

Una schematizzazione di una lista bidimensionale può essere la seguente (si noti che per ogni elemento della lista sono indicati due indici tra le parentesi quadre):

		Indice di colonna		
		0	1	2
Indice di riga	0	lista2D[0,0]	lista2D[0,1]	lista2D[0,2]
	1	lista2D[1,0]	lista2D[1,1]	lista2D[1,2]
	2	lista2D[2,0]	lista2D[2,1]	lista2D[2,2]
	3	lista2D[3,0]	lista2D[3,1]	lista2D[3,2]

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Come esercizio, si chiede di creare una lista bidimensionale come quella nella figura composta da 3 righe e 4 colonne, contenente i valori indicati:

		Indice di colonna			
		0	1	2	3
Indice di riga	0	**0**	**1**	**2**	**3**
	1	**4**	**5**	**6**	**7**
	2	**8**	**9**	**10**	**11**

Il programma che segue permette di inizializzare la lista bidimensionale come mostrato nella figura precedente. Vengono dichiarate due costanti globali RIGHE e COLONNE e nella funzione main(), si inizializza la lista bidimensionale come lista di tre liste ciascuna delle quali composta da 4 elementi uguali a zero. Attraverso una variabile contatore che verrà incrementata ad ogni ciclo del for più interno, si procederà con due cicli for annidati (uno che itererà sulle righe e uno, interno, che itererà sulle colonne) per la valorizzazione degli elementi della lista bidimensionale. Il programma si conclude con la stampa di default della lista bidimensionale.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
RIGHE = 3
COLONNE = 4

def main():

    valori = [[0,0,0,0],
              [0,0,0,0],
              [0,0,0,0]]

    a = 0
    for i in range(RIGHE):
        for j in range(COLONNE):
            valori[i][j] = a
            a+=1

    print(valori)

main()
```

La visualizzazione di default della lista bidimensionale sarà:

```
[[0, 1, 2, 3], [4, 5, 6, 7], [8, 9, 10, 11]]
```

Per stampare correttamente la lista bidimensionale, bisognerà ricorrere al seguente codice che provvederà a stampare una riga della lista bidimensionale per volta:

```
for i in range(RIGHE):
    print(valori[i])
```

La visualizzazione a schermo, infatti, sarà la seguente:

```
[0, 1, 2, 3]
[4, 5, 6, 7]
[8, 9, 10, 11]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

6. Tuple

Una **tupla** è una sequenza molto simile ad una lista, ma non modificabile. Una volta che una tupla viene creata, infatti, non può essere modificata. Il formato di una tupla differisce dalla lista perché per le tuple si usano le parentesi tonde (al posto delle quadre) come mostrato di seguito:

```
tupla = (elemento1, elemento2, ..)
```

Per le tuple, Python mette a disposizione le stesse operazioni delle liste, tranne quelle che modificano il contenuto che sono:

- append;
- remove;
- insert;
- reverse;
- sort.

Sarà, quindi, normale chiedersi quale sia il vantaggio di utilizzo delle tuple rispetto alle liste. I vantaggi delle tuple rispetto alle liste sono principalmente tre:

1. l'elaborazione delle tuple è più rapida rispetto all'elaborazione delle liste;
2. le tuple sono sicure in quanto non modificabili;
3. alcune operazioni in Python richiedono le tuple.

Esistono, infine, due **funzioni list()** e **tuple()** che permettono di convertire, rispettivamente, la tupla in lista:

```
lista = list(tupla)
```

e la lista in tupla:

```
tupla = tuple(lista)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).